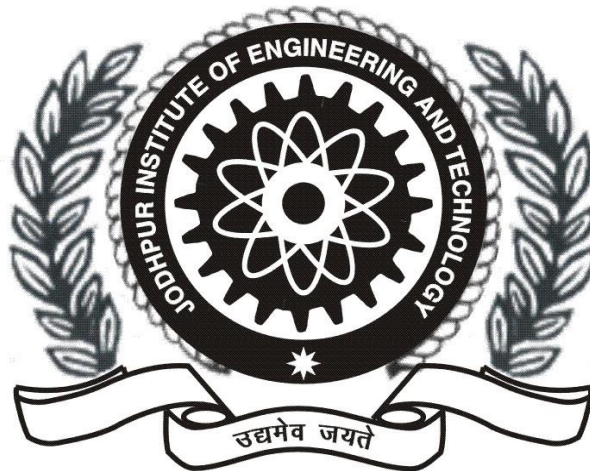


A
PROJECT REPORT ON
Robot collision Simulator (DSA)

In partial fulfillment of
B.Tech II yr (Computer Science & Engg.)



Submitted by:
RAJVEER SINGH
CSE, II Year

Submitted To:
Mrs. Harshita khanagarot
Assistant Professor

ACKNOWLEDGEMENT

I would like to acknowledge the contributions of the following people without whose help and guidance this project would not have been completed.

I respectfully thank **Mrs. harshit khangator** , for providing me an opportunity to do this project work and giving me all support and guidance, which made me complete the project up to very extent.

I am also thankful to **Prof Anju Jangid** HoD of Computer Science and Engineering Department, Jodhpur Institute of Engineering and Technology, for his/her constant encouragement, valuable suggestions and moral support and blessings.

Although it is not possible to name individually, I shall ever remain indebted to the faculty members of Jodhpur Institute of Engineering and Technology, for their persistent support and cooperation extended during his work.

This acknowledgement will remain incomplete if I fail to express our deep sense of obligation to my parents and God for their consistent blessings and encouragement.

TABLE OF CONTENTS

1. INTRODUCTION

2. TECHNOLOGY USED IN PROJECT

3. DETAILS OF PROJECT:

- 1. Functions/Modules Details**
- 2. Flow Chart**
- 3. Project Code**
- 4. Project Screenshots**

4. APPLICATIONS

5. CONCLUSION AND FUTURE WORK

6. REFERENCES

1. INTRODUCTION

The **Robot Collision Simulation** project investigates the interactions of multiple robots in a shared environment. Each robot is characterized by its position, health, and movement direction. The simulation models how these robots move toward each other and interact, focusing on collision outcomes based on their health statuses.

The primary objectives of this project are to:

1.Simulate Robot Behavior: Visualize how robots navigate and collide in a controlled environment.

2.Analyze Collision Dynamics: Understand the effects of health on survival outcomes during collisions.

By offering a dynamic representation of robot interactions, this project aims to deepen understanding of robotic behavior and the implications of physical interactions in real-world scenarios.

2. Technology used in Project

The **Robot Collision Simulation** project utilizes the following technologies:

1.Programming Language:

1. **Python:** Chosen for its simplicity and efficiency in development.

2.Graphical User Interface (GUI):

1. **Tkinter:** A built-in Python library for creating an interactive GUI that allows user input and visualization.

3.Data Structures:

1. **Lists:** For storing robot positions, health, and directions.
2. **Stacks:** Used for managing robot interactions during collisions.

4.Algorithms:

1. **Collision Detection:** Custom algorithms to identify collisions and resolve outcomes based on health.
2. **Simulation Logic:** Algorithms for updating robot positions and handling movement dynamics.

5.Visualization:

1. **Canvas Widget:** The Tkinter canvas is used for drawing and updating robot representations in real time.

These technologies work together to create an engaging simulation of robotic interactions.

Functions/Modules Details

1. Robot Class

- Handles the collision logic for the robots.
- **survived_robots_healths(positions, healths, directions)**: Determines surviving robots after collisions based on their health, position, and direction. Collisions occur between robots moving toward each other, and the robot with more health survives.

2. RobotUI Class

- Creates the GUI for user interaction using **Tkinter**.
- **__init__(self, master)**: Initializes the interface, including input fields, buttons, and a canvas for visualizing the simulation.
- **run_simulation()**: Validates input and starts the robot simulation, calling **animate_robots()**.
- **animate_robots(positions, healths, directions)**: Moves robots, checks for collisions, and updates the canvas to display robot movement.
- **resolve_collision(i, j, healths, directions)**: Resolves a collision between two robots by updating health and removing robots with zero health.
- **reset_simulation()**: Clears input, canvas, and resets buttons for a new simulation.
- **stop_simulation()**: Stops the current simulation.

3. Tkinter Module

- **Tk**: Main window for the GUI.
- **Entry**: Input fields for positions, directions, and healths.
- **Button**: Start, stop, and reset buttons.
- **Canvas**: Displays robot movements and collisions.
- **Label**: Shows messages like collisions and errors.

4. messagebox Module

- **showerror(title, message)**: Displays error messages for invalid input.

Project Code

1. Collision Resolution Algorithm

```
import tkinter as tk
from tkinter import messagebox, ttk

class Robot:
    def survived_robots_healths(self, positions, healths, directions):
        n = len(positions)
        indices = list(range(n))
        result = []
        stack = []

        # Sorting indices based on positions
        indices.sort(key=lambda x: positions[x])

        for current_index in indices:
            # Add right-moving robots to the stack
            if directions[current_index] == 'R':
                stack.append(current_index)
            else:
                while stack and healths[current_index] > 0:
                    # Pop the top robot from the stack for collision check
                    top_index = stack.pop()

                    # Top robot survives, current robot is destroyed
                    if healths[top_index] > healths[current_index]:
                        healths[top_index] -= 1
                        healths[current_index] = 0
                        stack.append(top_index)
                    elif healths[top_index] < healths[current_index]:
                        # Current robot survives, top robot is destroyed
                        healths[current_index] -= 1
                        healths[top_index] = 0
                    else:
                        # Both robots are destroyed
                        healths[current_index] = 0
                        healths[top_index] = 0

        # Collect surviving robots
        for index in range(n):
            if healths[index] > 0:
                result.append(healths[index])

        return result
```

2. Tkinter UI Setup

```
class RobotUI:
    def __init__(self, master):
        self.master = master
        master.title("Robot Collision Simulation")
        master.geometry("850x650")
        master.configure(bg="#F0F0F0")

        # Main title
        self.title_label = tk.Label(master, text="Robot Collision Simulation",
                                     font=("Helvetica", 18), bg="#F0F0F0")
        self.title_label.pack(pady=10)

        # Input fields for positions, directions, and healths
        self.positions_label = tk.Label(self.input_frame, text="Positions (space-separated):",
                                       bg="#F0F0F0")
        self.positions_entry = tk.Entry(self.input_frame, width=40)
        self.directions_label = tk.Label(self.input_frame, text="Directions (L/R):",
                                       bg="#F0F0F0")
        self.directions_entry = tk.Entry(self.input_frame, width=40)
        self.healths_label = tk.Label(self.input_frame, text="Healths (space-separated):",
                                       bg="#F0F0F0")
        self.healths_entry = tk.Entry(self.input_frame, width=40)

        # Control buttons for starting, resetting, and stopping the simulation
        self.submit_button = tk.Button(self.button_frame, text="Start Simulation",
                                       command=self.run_simulation, width=20, bg="#4CAF50", fg="white")
        self.reset_button = tk.Button(self.button_frame, text="Reset",
                                       command=self.reset_simulation, width=20, bg="#F44336", fg="white")
        self.stop_button = tk.Button(self.button_frame, text="Stop",
                                       command=self.stop_simulation, width=20, bg="#FFC107", fg="black",
                                       state=tk.DISABLED)

        # Canvas for visualization
        self.canvas = tk.Canvas(master, width=800, height=400, bg='white', bd=2,
                                relief="sunken")
        self.canvas.pack(pady=20)
```


3. Robot Movement and Collision Visualization

```
def animate_robots(self, positions, healths, directions):
    self.canvas.delete("all") # Clear previous drawings

    def move_robots():
        for i in range(len(positions)):
            if healths[i] <= 0:
                continue
            if directions[i] == 'R':
                positions[i] += 1
            elif directions[i] == 'L':
                positions[i] -= 1

        self.canvas.delete("all")
        for i in range(len(positions)):
            if healths[i] <= 0:
                continue

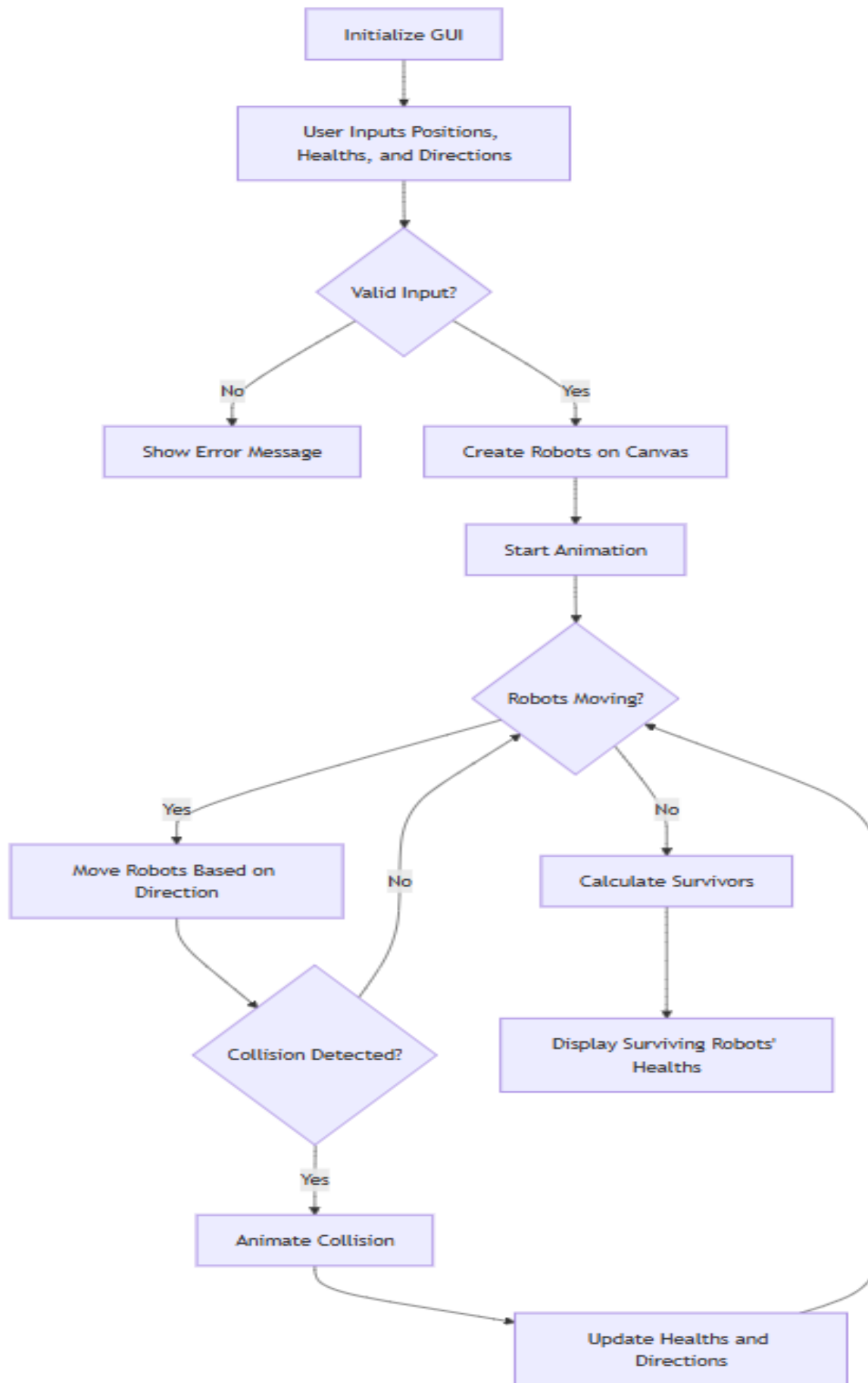
            x = positions[i] * 10
            y = 200
            color = self.get_health_color(healths[i])

            # Drawing robot body, eyes, health, and direction indicator
            self.canvas.create_rectangle(x, y, x + 30, y + 40, fill=color, outline="black")
            self.canvas.create_oval(x + 5, y + 5, x + 10, y + 10, fill="black")
            self.canvas.create_oval(x + 20, y + 5, x + 25, y + 10, fill="black")
            self.canvas.create_text(x + 15, y + 20, text=str(healths[i]), fill="black")
            if directions[i] == 'R':
                self.canvas.create_text(x + 15, y + 50, text="→", fill="black")
            else:
                self.canvas.create_text(x + 15, y + 50, text="←", fill="black")

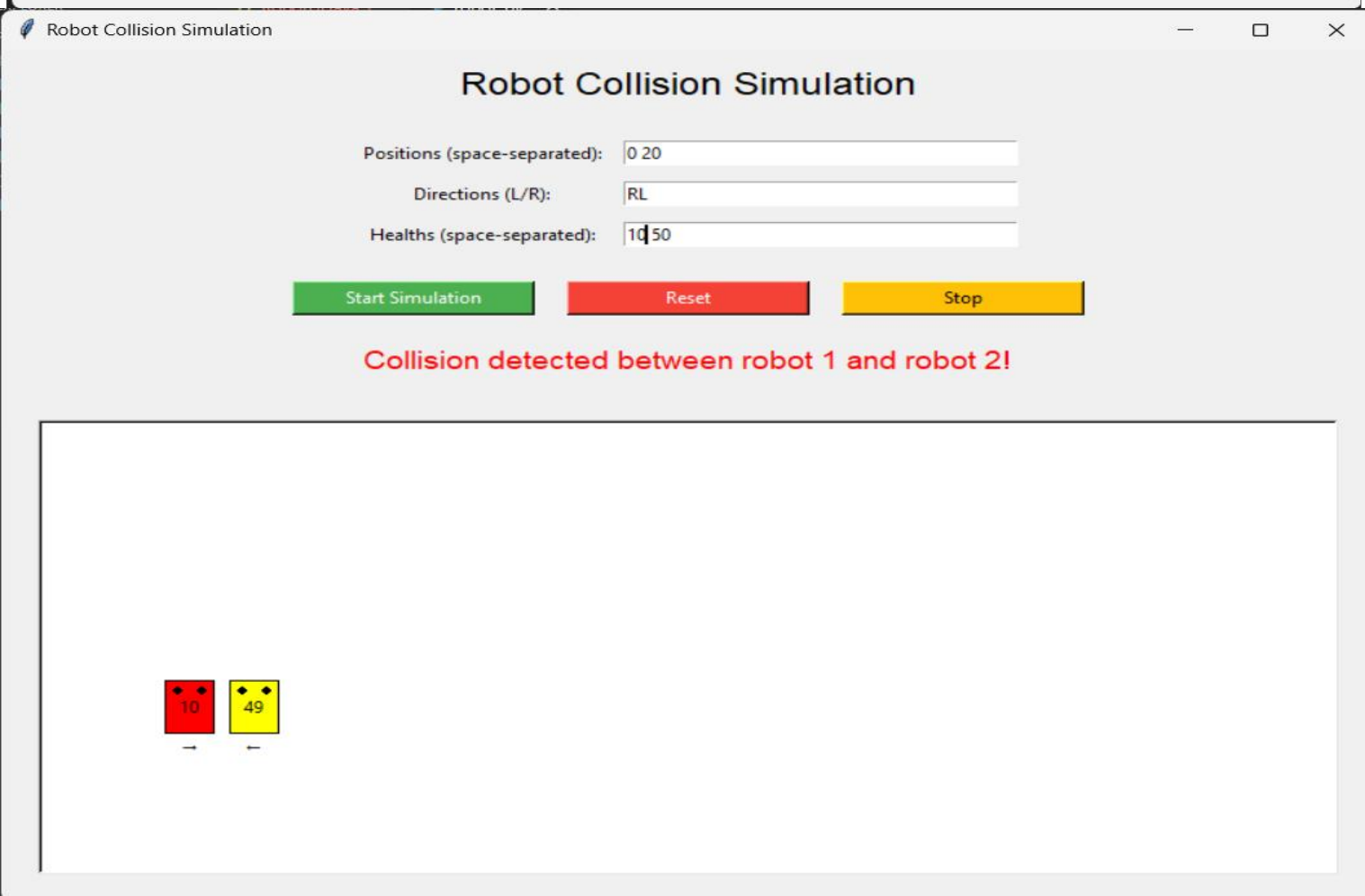
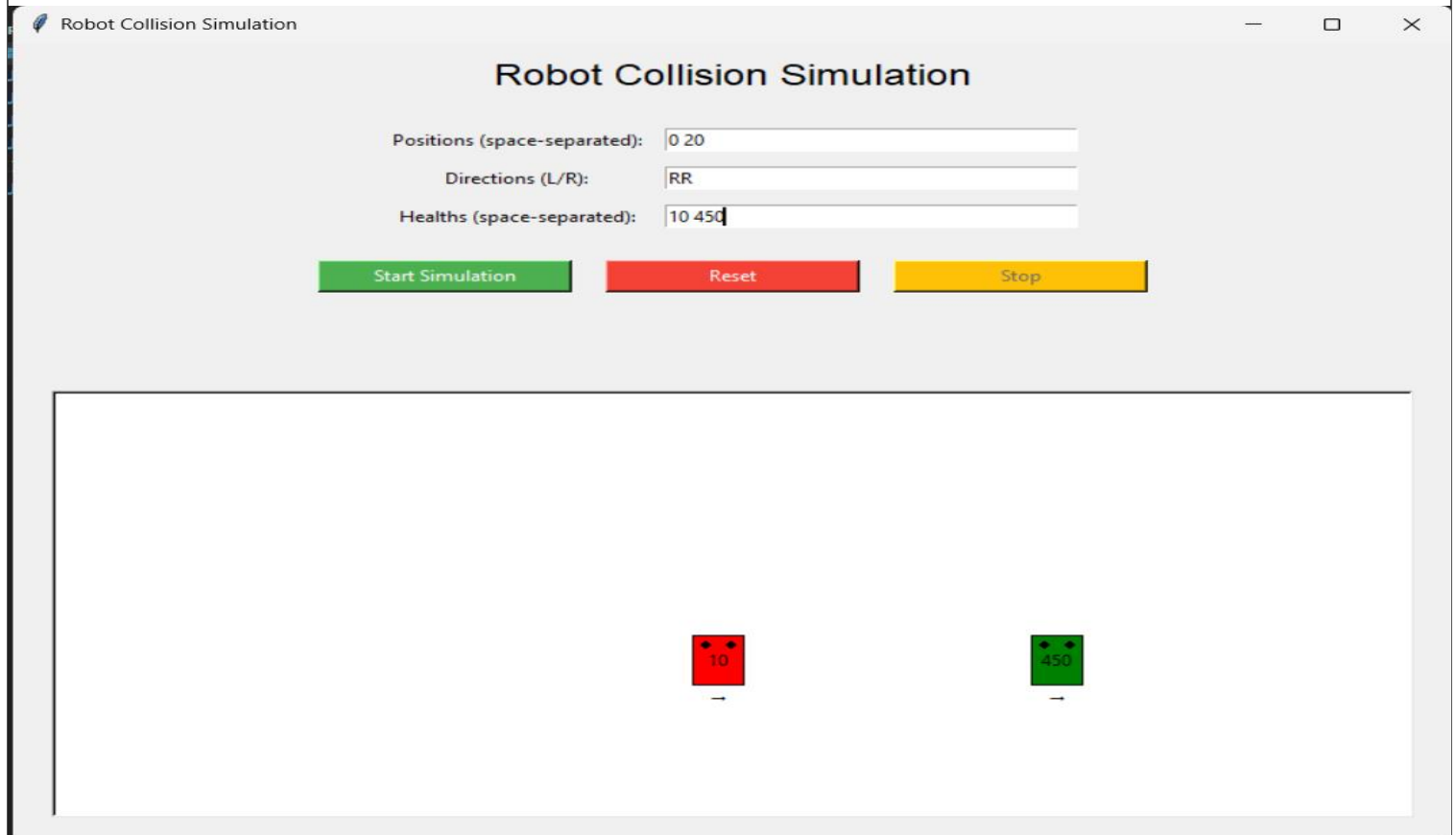
        # Collision detection and resolution
        for j in range(i + 1, len(positions)):
            if directions[i] == 'R' and directions[j] == 'L' and positions[i] < positions[j]:
                if abs(positions[i] - positions[j]) <= 5:
                    self.resolve_collision(i, j, healths, directions)
                    self.display_collision_message(i, j)

        self.timer = self.canvas.after(100, move_robots)
    move_robots()
```

Flow chart:



SCREEN SHOT



4. APPLICATIONS

The **Robot Collision Simulation** project has several practical applications, including:

1.Educational Tool:

- The simulation serves as a hands-on learning resource for students studying robotics, programming, and physics, helping them understand complex concepts related to movement and collisions.

2.Robotics Research:

- Researchers can use the simulation to analyze and test various algorithms for collision detection and resolution in robotic systems, facilitating the development of more advanced robotic behaviors.

3.Game Development:

- The principles applied in this project can be adapted for game development, where collision detection and resolution are crucial for creating realistic interactions between game characters or objects.

4.Autonomous Systems:

- Insights from the simulation can inform the design of autonomous robots, particularly in scenarios involving navigation in crowded or dynamic environments.

5.Real-World Applications:

- The concepts of collision dynamics can be applied in fields such as transportation, logistics, and manufacturing, where understanding interactions between automated systems is essential for safety and efficiency.

By demonstrating robotic interactions and decision-making processes, the project contributes to both academic and practical advancements in the field of robotics.

5. FUTURE WORKS AND CONCLUSIONS

Future Work:

- There are several potential enhancements for this project:
- **Advanced Collision Detection:** Improve collision detection by considering more complex movement patterns, such as diagonal or multi-dimensional movement.
- **Different Robot Abilities:** Introduce different robot types with unique abilities or behaviors, adding complexity to the collision logic.
- **Obstacle Incorporation:** Add obstacles to the canvas, forcing robots to navigate around them and increasing the complexity of the simulation.
- **Performance Optimization:** Improve the algorithm's efficiency to handle larger numbers of robots in the simulation.
- **Simulation Analytics:** Introduce detailed analytics or statistical insights (e.g., survival rates, collision frequency) to provide users with more information about each simulation.

Conclusion:

- In this project, we developed a robot collision simulation using Tkinter for a graphical interface and implemented a collision resolution algorithm. The system allows users to input positions, directions, and health values for robots and visually simulates their movements and collisions. The stack-based approach efficiently handles robot collisions, determining which robots survive based on health values. This project successfully demonstrates an interactive and dynamic way to simulate real-time events like robot collisions while providing clear feedback to users.

6. REFERENCES

➤ **Tkinter Documentation**

Python Software Foundation. (n.d.). Tkinter — Python interface to Tcl/Tk. Retrieved from <https://docs.python.org/3/library/tkinter.html>

➤ **Greedy Algorithm Overview**

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Chapter 16: Greedy Algorithms.

➤ **Python Stack Usage**

Python Software Foundation. (n.d.). List as Stack Operations in Python. Retrieved from <https://docs.python.org/3/tutorial/datastructures.html>

➤ **Collision Detection in Games**

Millington, I., & Funge, J. (2009). *Artificial Intelligence for Games* (2nd ed.). CRC Press. Chapter 8: Collision Detection.

➤ **Graphical User Interfaces with Tkinter**

Grayson, J. (2000). *Python and Tkinter Programming*. Manning Publications.